

Satida Sookpong

2-1 Word Segmentation, POS Tagging & Sequence Labeling

1323326 การประมวลผลภาษาธรรมชาติ (Natural Language Processing)
1-0303 Computer & Sound; Lab 2



Word Segmentation

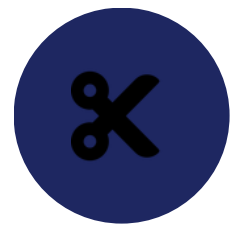


POS Tagging

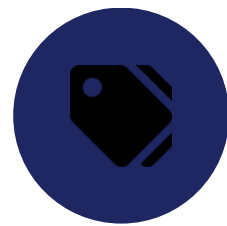


Sequence Labeling

What We'll Cover

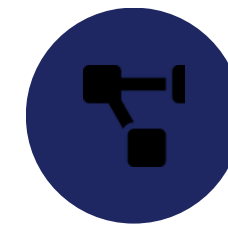


1. Word Segmentation



2. POS Tagging

- part of speech
- knowledge-based tagging
- HMM
- Viterbi decoding
- CRF



3. Sequence Labeling

Named Entity Recognition (NER)

Word Segmentation

Why we need Word Segmentation?

English: "I go to school"

Thai: "ฉันไปโรงเรียน"

In English, spaces mark word boundaries. But languages like Thai, Chinese, and Japanese are written without spaces, so the boundaries must be inferred:



ไม่มีตัวคั่นคำ

ภาษาไทย จีน ญี่ปุ่น เขียนติดกัน
ไม่มี whitespace ระหว่างคำเหมือน
อังกฤษ



คำกำกวม (Ambiguity)

สายอักขระเดียวกันตัดคำได้หลายแบบ
ให้ความหมายต่างกันโดยสิ้นเชิง เช่น
"ตากลม"



ไม่มีนิยาม "คำ" ที่ตายตัว

Word Segmentation

Segmentation Approaches



Dictionary-based Matching

Match longest / shortest known words against a lexicon (e.g., maximum matching).



Rule-based Heuristics

Hand-crafted linguistic rules resolve ambiguous boundaries.



Statistical / ML-based

Character-level classifiers or sequence models predict boundaries. eg.HMM, CRF, Deep Learning (LSTM/Transformer)

FMM: Example

แนวคิด: เริ่มจากตำแหน่งซ้ายสุด พยายามจับคู่สายอักขระที่ "ยาวที่สุด" กับคำในพจนานุกรมก่อนเสมอ ถ้าไม่พบให้ลดความยาวลงทีละ 1 ตัวอักษร

พจนานุกรม (ย่อ) :
{นัก, นักเรียน, เรียน, ไป,
โรง, โรงเรียน, ...}

ประโยค: นั-ก-เ-ร-ี-ย-น-ไ-ป-โ-ร-ง-เ-ร-ี-ย-น (18 ตัวอักษร ไม่มีช่องว่าง)

Step 1

ลอง "นักเรียนไป" (8) ✗ → ... → ลอง "นักเรียน" (4) ✓

ตัด: นักเรียน

Step 2

ลอง "ไปโรงเรียน" (4) ✗ → ... → ลอง "ไป" (1) ✓

ตัด: ไป

Step 3

ลอง "โรงเรียน" (4) ✓

ตัด: โรงเรียน

ผลลัพธ์: นักเรียน | ไป | โรงเรียน (ตรวจสอบด้วยโปรแกรม Python ได้ผลลัพธ์ตรงกัน)

Backward Maximum Matching (BMM): แนวคิดเดียวกัน แต่เริ่มจากขวาสุดแล้วไล่มาซ้าย

Maximum Matching

ตา + กลม = round eyes vs. ตาก + ลม = to air out

Forward Maximum Matching (FMM)

i=0: ลอง "ตากลม" (5) ✗
 ลอง "ตากล" (4) ✗
 ลอง "ตาก" (3) ✓ พบ! -> ตัด "ตาก"
i=3: ลอง "ลม" (2) ✓ พบ! -> ตัด "ลม"

ผลลัพธ์ FMM: ตาก | ลม

Backward Maximum Matching (BMM)

j=5: ลอง "ตากลม" (5) ✗
 ลอง "ากลม" (4) ✗
 ลอง "กลม" (3) ✓ พบ! -> ตัด "กลม"
j=2: ลอง "ตา" (2) ✓ พบ! -> ตัด "ตา"

ผลลัพธ์ BMM: ตา | กลม

Statistic Segmentation

แนวคิด

- แทนที่จะเลือกคำ "ยาวที่สุด" แบบโลภ ให้เลือก "ลำดับการตัดคำที่มีความน่าจะเป็นสูงสุด"
- ใช้ unigram/bigram frequency จาก corpus ขนาดใหญ่ ประมาณค่าความน่าจะเป็นของแต่ละคำ
- สูตร (unigram): $\text{argmax} \prod P(w_i)$ สำหรับทุกวิธีตัดคำที่เป็นไปได้
- เทียบเท่าแนวคิดเดียวกับ N-gram Language Model ที่เรียนในสัปดาห์ที่แล้ว!

$P(\text{ตา}) \times P(\text{กลม})$ เทียบกับ $P(\text{ตาก}) \times P(\text{ลม})$

- นับความถี่ของแต่ละคำใน corpus ตัวอย่าง (unigram frequency)
- คำนวณความน่าจะเป็นของ "ตา+กลม" เทียบกับ "ตาก+ลม"
- เลือกลำดับที่ให้ผลคูณความน่าจะเป็นสูงกว่า
- → ดูตัวอย่างคำนวณจริงในสไลด์ถัดไป

Prob 101

1. Classical Probability (นิยามพื้นฐาน)

$$P(E) = \frac{N(E)}{N(S)}$$

- $N(E)$ = จำนวนกรณีที่เกิดเหตุการณ์ E เกิดขึ้น
- $N(S)$ = จำนวนกรณีทั้งหมดใน sample space

2. Conditional Probability (ความน่าจะเป็นแบบมีเงื่อนไข)

$$P(A | B) = \frac{P(A \cap B)}{P(B)}$$

"ความน่าจะเป็นของ A เมื่อรู้ว่า B เกิดขึ้นแล้ว"

3. Chain Rule (กฎลูกโซ่)

$$P(A_1, A_2, \dots, A_n) = P(A_1) \cdot P(A_2 | A_1) \cdot P(A_3 | A_1, A_2) \dots$$

4. Independence (ความเป็นอิสระ)

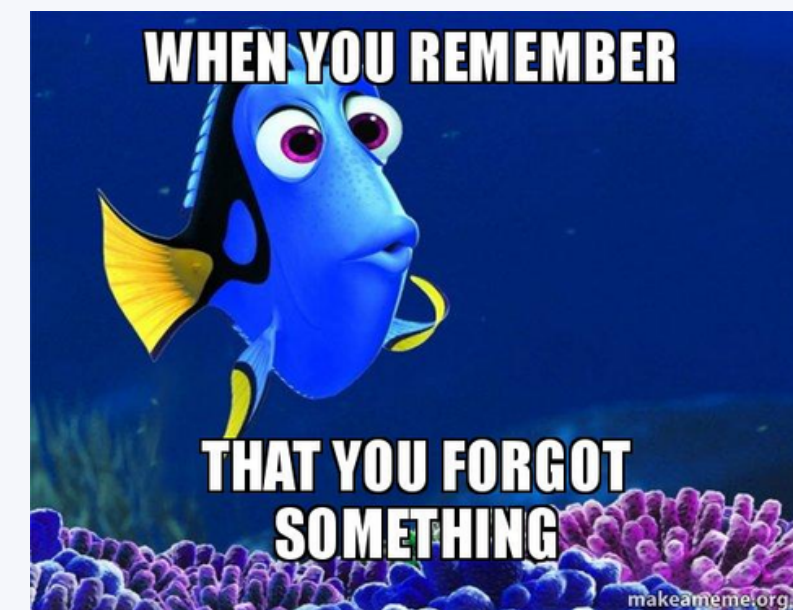
$$P(A \cap B) = P(A) \cdot P(B) \quad (\text{ถ้า } A, B \text{ เป็นอิสระต่อกัน})$$

5. Bayes' Theorem (ทฤษฎีบทของเบย์ส)

$$P(A | B) = \frac{P(B | A) \cdot P(A)}{P(B)}$$

6. Law of Total Probability / Marginalization

$$P(A) = \sum_i P(A | B_i) \cdot P(B_i)$$



WORKED EXAMPLE

Statistical Segmentation

ความถี่คำจาก corpus ตัวอย่าง (toy corpus) → จำนวนคำทั้งหมด = 500+80+60+200 = 840

คำ	ตา	กลม	ตาก	ลม
ความถี่ (count)	500	80	60	200
P(w) = count/840	0.595	0.095	0.071	0.238

$$P(\text{ตา+กลม}) = 0.595 \times 0.095$$
$$= 0.0567$$

$$P(\text{ตาก+ลม}) = 0.071 \times 0.238$$
$$= 0.0170$$

0.0567 > 0.0170 → Statistical Segmentation เลือก “ตา + กลม”

สรุปเปรียบเทียบวิธีตัดคำภาษาไทย

วิธี	แนวคิดหลัก	ข้อจำกัด / จุดเด่น
Maximum Matching (Forward/Backward)	จับคู่คำยาวที่สุดกับพจนานุกรมแบบโลภ (greedy)	เร็ว ไม่ต้อง train แต่แก้ความกำกวมไม่ได้ดี ขึ้นกับพจนานุกรม
Statistical Segmentation	เลือกลำดับการตัดคำที่ให้ค่าความน่าจะเป็น (n-gram) สูงสุด	แก้ความกำกวมได้ดีกว่า ต้องมี corpus สถิติที่น่าเชื่อถือ
HMM / CRF (สถิติเชิงลำดับ)	มองการตัดคำเป็น sequence labeling	แม่นยำขึ้น ใช้ training data ที่ label แล้ว
Deep Learning (BiLSTM / Transformer)	เรียนรู้ลักษณะคำจากข้อมูลจำนวนมากโดยตรง (เช่น deepcut, PyThaiNLP)	แม่นยำสูงสุดในปัจจุบัน แต่ต้องใช้ข้อมูล/ทรัพยากรมาก

POS Tagging

Part-of-Speech Tagging



POS Tagging

- กระบวนการกำหนด "Part of Speech" (noun, verb, adjective, ...) ให้กับทุกคำในประโยค
- ต่างจาก tokenization/segmentation: ตัดคำแล้ว POS Tagging บอกว่าแต่ละคำ "ทำหน้าที่อะไร" ในประโยค
- เป็นพื้นฐานสำคัญของงานขั้นสูง เช่น Parsing, Named Entity Recognition, Information Extraction, Machine Translation
- คำเดียวกันอาจมีหลายหมวดคำได้ขึ้นกับบริบท เช่น "back" เป็นได้ทั้ง VB (กลับ), NN (หลัง), JJ, RB

ตัวอย่าง

Janet	will	back	the	bill
NNP	MD	VB	DT	NN
(ชื่อ)	(กริยาช่วย)	(กริยา)	(นำหน้านาม)	(นาม)

ทำไมยาก?

"back" อาจเป็น VB (สนับสนุน/ถอยกลับ), NN (แผ่นหลัง), JJ (ข้างหลัง) หรือ RB — ต้องดูบริบทถึงจะรู้ว่าเป็นหมวดคำใด

No	POS Tag	Description	Example	No	POS Tag	Description	Example
1	CC	coordinating conjunction	and, but, or	24	SYM	Symbol	\$ / [= *
2	CD	cardinal number	1, third	25	TO	infinitive 'to'	to
3	DT	determiner	a, the	26	UH	interjection	haha, oops
4	EX	existential there	there is	27	VB	verb - base form	drink
5	FW	foreign word	les	28	VBD	verb - past tense	drank
6	IN	preposition, sub-conj	in, of, by, like	29	VBG	verb - gerund	drinking
7	JJ	adjective	big, wide, green	30	VBN	verb - past participle	drunk
8	JJR	adjective, comparative	bigger, wider, greener	31	VBP	verb - non-3sg pres	drink
9	JJS	adjective, superlative	biggest, wildest, greenest	32	VBZ	verb - 3sg pres	drinks
10	LS	list marker	1), One, i	33	WDT	wh-determiner	which, that
11	MD	modal	can, could, shall, will	34	WP	wh-pronoun	who, what
12	NN	noun, singular or mass	table, shop	35	WP\$	possessive wh-pronoun	whose, those
13	NNS	noun plural	tables, shops	36	WRB	wh-abverb	where, when, how
14	NNP	proper noun, singular	Samsung	37	#	#	#
15	NNPS	proper noun, plural	Vikings	38	\$	\$	\$
16	PDT	predeterminer	all/both the students	39	"	Left quotation	' '
17	POS	possessive ending	friend's	40	"	right quotation	' '
18	PP	personal pronoun	I, he, it, you	41	(	Opening brackets	{ {
19	PPZ	possessive pronoun	my, his, your, one's	42	)	Closing brackets	} }
20	RB	adverb	however, quickly, here	43	,	Comma	,
21	RBR	adverb, comparative	better, quicker	44	:	Sent-final punc	. ! ?
22	RBS	adverb, superlative	best, quickest	45	:	Mid-sentence punc	: ; ... -
23	RP	particle	of, up (e.g. give up)				

Fig. 3.3 Penn Treebank POS Tags (with punctuation)



POS Tagset: ชุดหมวดคำมาตรฐาน

Tag	ความหมาย	ตัวอย่าง
NOUN	คำนาม	algorithm, cat, mango
VERB	คำกริยา	draw, provide, go
ADJ	คำคุณศัพท์	red, young, awesome
ADV	คำวิเศษณ์	very, slowly, yesterday
PROPN	คำนามเฉพาะ	Regina, IBM, Colorado
DET	คำนำหน้านาม	a, an, the, this
ADP	คำบุพบท	in, on, by, under
PRON	คำสรรพนาม	she, who, I

Penn Treebank tagset (ละเอียดยกว่า UD ~45 tags เช่น NNP, MD, VB, DT, NN, JJ, RB)
ภาษาไทยมี **ORCHID tagset** (NECTEC, ~47 tags เช่น NCMN=คำนามทั่วไป, VACT=กริยาแสดงการกระทำ)

แนวทางการทำ POS Tagging



Rule-based

ใช้พจนานุกรม + กฎไวยากรณ์ที่มนุษย์เขียนขึ้น เพื่อกำจัดความกำกวมของหมวดคำ



Stochastic / Statistical

ใช้ความถี่/ความน่าจะเป็นจาก corpus ที่ label ไว้แล้ว (เช่น HMM) ตัดสินหมวดคำที่เป็นไปได้มากที่สุด



Hybrid (TBL / Brill Tagger)

ผสาน stochastic (เดาเริ่มต้น) เข้ากับ rule-based (ปรับแก้ด้วยกฎ) เพื่อความแม่นยำสูงขึ้น

กระบวนการทั่วไป (General Pipeline): รับประโยคที่ตัดคำแล้ว (tokens/words) → ค้นหาหมวดคำที่เป็นไปได้ทั้งหมดของแต่ละคำจากพจนานุกรม → ใช้กฎหรือแบบจำลองทางสถิติเลือกหมวดคำที่ถูกต้องที่สุดตามบริบท → ได้ลำดับหมวดคำ (tag sequence) เป็นผลลัพธ์

1.Rule-based Tagging

How?

- **1. Dictionary lookup:** ค้นพจนานุกรมเพื่อหา "หมวดคำที่เป็นไปได้ทั้งหมด" ของแต่ละคำ (คำหนึ่งอาจมีได้หลายหมวดคำ)
- **2. Disambiguation using rules:** ใช้กฎไวยากรณ์ (hand-written rules) ที่พิจารณาริบทของคำข้างเคียง เพื่อเลือกหมวดคำที่ถูกต้อง
- แหล่งความรู้เพิ่มเติม:
 - กฎของรูปคำ (-ed → past tense, -ing → verb ต่อเนื่อง, -ly → adverb)
 - ตัวพิมพ์ใหญ่ (capitalization → proper noun)

ตัวอย่างกฎ (Rule Example)

a book
a -> DET (ชัดเจน)
book -> NOUN หรือ VERB? (กำกวม)

Rule: ถ้าคำก่อนหน้าเป็น DET
-> คำถัดไปต้องเป็น NOUN
ดังนั้น "book" -> NOUN

ข้อดี: อธิบายได้ (explainable), แม่นยำสูงถ้ากฎครอบคลุม

ข้อเสีย: เขียนกฎเองใช้แรงงานมาก และปรับ scale ยาก

2. Stochastic / Statistical Tagging

- ใช้ "**ความถี่**" (frequency) ของหมวดคำและลำดับหมวดคำจาก corpus ที่ label ไว้แล้ว (tagged corpus)
 - เลือกหมวดคำที่พบบ่อยที่สุดของคำนั้น (ไม่ดูบริบท)
 - Unigram/Most-Frequent-Tag Baseline
 - พิจารณาลำดับหมวดคำ (tag sequence) ทั้งประโยคพร้อมกัน
 - Hidden Markov Model (HMM)
- ข้อดี: เรียนรู้จากข้อมูลอัตโนมัติ ไม่ต้องเขียนกฎเอง
- ข้อเสีย: ต้องมี tagged corpus ขนาดใหญ่



Statistical Approaches

1



**Hidden Markov Model
(HMM)**

2



Viterbi Decoding

3



**Conditional Random Field
(CRF)**

Markov chain

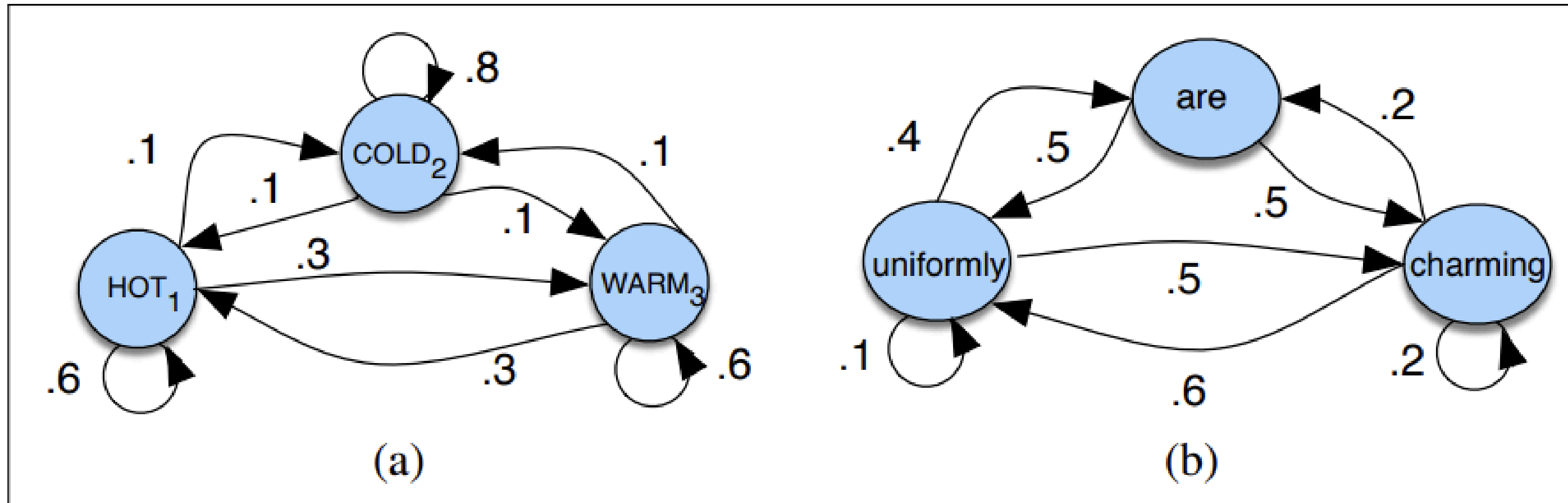


Figure 17.8 A Markov chain for weather (a) and one for words (b), showing states and transitions. A start distribution π is required; setting $\pi = [0.1, 0.7, 0.2]$ for (a) would mean a probability 0.7 of starting in state 2 (cold), probability 0.1 of starting in state 1 (hot), etc.

HIDDEN MARKOV MODEL

HMM

Q

set of N states - ศัพท์คำ (POS tags) ทั้งหมด

A

transition probability matrix : $P(\text{tag}_i | \text{tag}_{i-1})$ ความน่าจะเป็นที่ย้ายจาก tag $i-1 \rightarrow \text{tag } i$

B

observation likelihoods: $P(\text{word}_i | \text{tag}_i)$ ความน่าจะเป็นที่ tag จะปรากฏเป็นคำนั้น

π

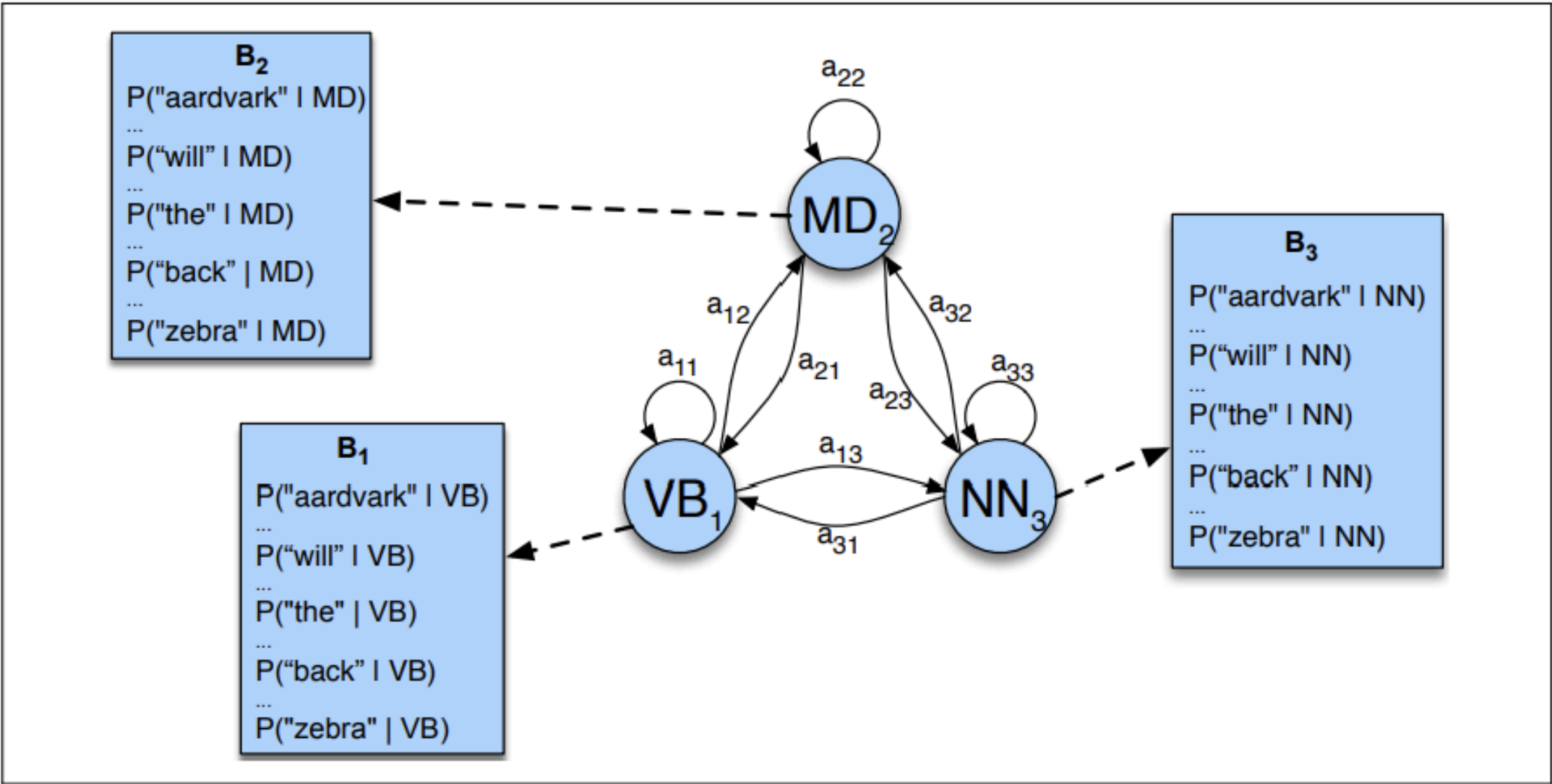
initial distribution — π_i = ความน่าจะเป็นเริ่มต้นใน tag i (tag แรกของประโยค)

The HMM is given as input $O = o_1 o_2 \dots o_T$: a sequence of T **observations**, each one drawn from the vocabulary V .

Markov Assumption: $P(q_i | q_1, \dots, q_{i-1}) = P(q_i | q_{i-1})$ (17.6)

Output Independence: $P(o_i | q_1, \dots, q_i, \dots, q_T, o_1, \dots, o_i, \dots, o_T) = P(o_i | q_i)$ (17.7)

HMM



Observation: ลำดับคำ
Hidden State: POS

Figure 17.9 An illustration of the two parts of an HMM representation: the A transition probabilities used to compute the prior probability, and the B observation likelihoods that are associated with each state, one likelihood for each possible observation word.

HMM tagging as Decoding

5. Bayes' Theorem (ทฤษฎีบทของเบย์ส)

$$P(A | B) = \frac{P(B | A) \cdot P(A)}{P(B)}$$

(1) $\hat{t} = \operatorname{argmax} P(t_1 \dots t_n | w_1 \dots w_n)$

เป้าหมาย: หา *tag sequence* ที่น่าจะเป็นที่สุด

(2) $\hat{t} = \operatorname{argmax} P(w_1 \dots w_n | t_1 \dots t_n) P(t_1 \dots t_n)$

ใช้ *Bayes' Rule* ($P(w)$ คงที่)

(3) $P(w_1 \dots w_n | t_1 \dots t_n) \approx \prod P(w_i | t_i)$

สมมติฐาน *Output Independence*

(4) $P(t_1 \dots t_n) \approx \prod P(t_i | t_{i-1})$

สมมติฐาน *Markov Assumption*

$$\hat{t}_{1:n} = \operatorname{argmax}_{t_1 \dots t_n} P(t_1 \dots t_n | w_1 \dots w_n) \approx \operatorname{argmax}_{t_1 \dots t_n} \prod_{i=1}^n \overbrace{P(w_i | t_i)}^{\text{emission}} \overbrace{P(t_i | t_{i-1})}^{\text{transition}} \quad (17.17)$$

HMM Tables: “Janet will back the bill”

A: tag transition probabilities $P(t_i \mid t_{i-1})$ (แถว = tag ก่อนหน้า, คอลัมน์ = tag ถัดไป)

	NNP	MD	VB	JJ	NN	RB	DT
<s>	.28	.0006	.0031	.045	.045	.051	.203
NNP	.378	.011	.0009	.0084	.058	.009	.0025
MD	.0008	.0002	.797	.0005	.0008	.170	.0041
VB	.032	.0005	.005	.084	.062	.051	.223

B: observation likelihoods $P(\text{word} \mid \text{tag})$ (สังเกต: “Janet” เป็นได้แค่ NNP เท่านั้น)

	Janet	will	back	the	bill
NNP	.000032	0	0	.000048	0
MD	0	.308431	0	0	0
VB	0	.000028	.000672	0	.000028
NN	0	.000200	.000223	0	.002337
DT	0	0	0	.506099	0

ที่มา: *WSJ Corpus (Speech and Language Processing, Fig. 17.12–17.13)*

Viterbi Algorithm - Decoding

แนวคิดหลัก

- Dynamic Programming (DP) to find **Hidden State**
- value of each cell $v(j)$: **recursively** taking the most **probable path** that could lead us to this cell
- "lattice/trellis": ตาราง N แถว (tags) $\times$ T คอลัมน์ (คำในประโยค)
- หาความน่าจะเป็นของเส้นทางที่ดีที่สุดที่มาถึง tag j ณ ตำแหน่ง t
- เก็บ **backpointer** ทุกเซลล์ เพื่อย้อนรอย (backtrace) คลำดับ tag ที่ดีที่สุดทั้งประโยคเมื่อคำนวณจบ

Given HMM λ

$$v_t(j) = \max_{q_1, \dots, q_{t-1}} P(q_1 \dots q_{t-1}, o_1, o_2 \dots o_t, q_t = j | \lambda) \quad (17.18)$$

$$v_t(j) = \max_{i=1}^N v_{t-1}(i) a_{ij} b_j(o_t) \quad (17.19)$$

$v_{t-1}(i)$	the previous Viterbi path probability from the previous time step
a_{ij}	the transition probability from previous state q_i to current state q_j
$b_j(o_t)$	the state observation likelihood of the observation symbol o_t given the current state j

	NNP	MD	VB	JJ	NN	RB	DT
$\langle s \rangle$	0.2767	0.0006	0.0031	0.0453	0.0449	0.0510	0.2026
NNP	0.3777	0.0110	0.0009	0.0084	0.0584	0.0090	0.0025
MD	0.0008	0.0002	0.7968	0.0005	0.0008	0.1698	0.0041
VB	0.0322	0.0005	0.0050	0.0837	0.0615	0.0514	0.2231
JJ	0.0366	0.0004	0.0001	0.0733	0.4509	0.0036	0.0036
NN	0.0096	0.0176	0.0014	0.0086	0.1216	0.0177	0.0068
RB	0.0068	0.0102	0.1011	0.1012	0.0120	0.0728	0.0479
DT	0.1147	0.0021	0.0002	0.2157	0.4744	0.0102	0.0017

Figure 17.12 The A transition probabilities $P(t_i|t_{i-1})$ computed from the WSJ corpus without smoothing. Rows are labeled with the conditioning event; thus $P(VB|MD)$ is 0.7968. $\langle s \rangle$ is the start token.

	Janet	will	back	the	bill
NNP	0.000032	0	0	0.000048	0
MD	0	0.308431	0	0	0
VB	0	0.000028	0.000672	0	0.000028
JJ	0	0	0.000340	0	0
NN	0	0.000200	0.000223	0	0.002337
RB	0	0	0.010446	0	0
DT	0	0	0	0.506099	0

Figure 17.13 Observation likelihoods B computed from the WSJ corpus without smoothing, simplified slightly.

WORKED EXAMPLE

Viterbi: คอลัมน์ที่ 1-2 (“Janet”, “will”)

Column 1 — “Janet”: $v1(j) = \pi_j \times b_j(\text{Janet})$ (เฉพาะ NNP ที่ไม่เป็นศูนย์ เพราะ “Janet” ปรากฏเป็น NNP เท่านั้นใน corpus)

$$v1(NNP) = \pi(NNP) \times b(\text{Janet} | NNP) = 0.28 \times 0.000032 = 8.854e-6 \quad \leftarrow \text{ค่าเดียวที่ไม่เป็นศูนย์}$$
$$v1(MD) = v1(VB) = v1(JJ) = v1(NN) = v1(RB) = v1(DT) = 0 \quad (\text{เพราะ } b(\text{Janet} | \text{tag เหล่านี้}) = 0)$$

Column 2 — “will”:

$$v_t(j) = \max_{i=1}^N v_{t-1}(i) a_{ij} b_j(o_t) \tag{17.19}$$

Tag j	$v1(NNP) \times a(NNP, j)$	$\times b_j(\text{will})$	= v2(j)	backpointer
MD	$8.854e-6 \times 0.011 = 9.74e-8$	$\times 0.308431$	3.004e-8	NNP
VB	$8.854e-6 \times 0.0009 = 7.97e-9$	$\times 0.000028$	2.231e-13	NNP
NN	$8.854e-6 \times 0.058 = 5.14e-7$	$\times 0.000200$	1.034e-10	NNP
อื่น ๆ (NNP, JJ, RB, DT)	—	$\times 0$	0	NNP

สูงสุดในคอลัมน์ 2 คือ $v2(MD) = 3.004 \times 10^{-8} \rightarrow$ ตอบ will เป็น MD (กริยาช่วย)

WORKED EXAMPLE

Viterbi: คำนวณจนจบประโยค + Backtrace

ทำซ้ำ recurrence เดิมสำหรับคอลัมน์ back, the, bill จนครบ 5 คำ ผลลัพธ์ค่า v สูงสุดของแต่ละคอลัมน์ (คำนวณเต็มด้วย Python):

คำ	Janet	will	back	the	bill
best tag	NNP	MD	VB	DT	NN
best v (×10 ⁿ)	8.854e-6	3.004e-8	1.609e-11	1.816e-12	2.014e-15
backpointer	(start)	NNP	MD	VB	DT

Backtrace จากคอลัมน์สุดท้าย (bill → NN สูงสุด) ตามลูกศร backpointer ย้อนกลับ:

Janet/NNP → will/MD → back/VB → the/DT → bill/NN

Computational Complexity ของ Viterbi

Brute-force (ลองทุกความเป็นไปได้)

จำนวน tag sequence ที่เป็นไปได้ = T^N

$$O(T^N)$$

N = ความยาวประโยค (จำนวนคำ), T = จำนวนหมวดคำ

Viterbi (Dynamic Programming)

เติมตาราง $N \times T$ ทีละเซลล์ แต่ละเซลล์เทียบ T ทางเลือกก่อนหน้า

$$O(N \times T^2)$$

เร็วกว่า Brute-force แบบเอ็กซ์โพเนนเชียลอย่างมาก

ตัวอย่างตัวเลขจริง: ประโยค "Janet will back the bill" ($N=5$ คำ) และ 7 หมวดคำ ($T=7$)

Brute-force: $T^N = 7^5 = 16,807$ ลำดับ Viterbi: $N \times T^2 = 5 \times 7^2 = 245$ การคำนวณ

เร็วขึ้นประมาณ 68.6 เท่า

Conditional Random Fields (CRF)

ทำไมต้องมี CRF ถ้ามี HMM แล้ว?

- HMM เป็น Generative Model: จากโมเดล $P(W|T)P(T)$ แล้วใช้ Bayes เพื่อหา $P(T|W)$
- HMM มองคำได้ที่ละคำเดียว: eg. look $\rightarrow$ looking
- HMM มองแค่คำก่อนหน้า ทำให้จับความสัมพันธ์นอกจากนั้นไม่ได้
- CRF เป็น Discriminative Model: คำนวณ $P(T|W)$ โดยตรง จึงใส่ **arbitrary features** ได้อย่างอิสระ
- เป็น linear-chain CRF: log-linear model เหมือน logistic regression แต่ขยายไปทั้งลำดับ (sequence)

$$\hat{Y} = \operatorname{argmax}_{Y \in \mathcal{Y}} P(Y|X) \quad (17.22)$$

$$p(Y|X) = \frac{1}{Z(X)} \exp \left(\sum_{k=1}^K w_k F_k(X, Y) \right) \quad (17.24)$$

$$Z(X) = \sum_{Y' \in \mathcal{Y}} \exp \left(\sum_{k=1}^K w_k F_k(X, Y') \right) \quad (17.25)$$

w = weight of each feature

$Z(W)$ = normalization term

$F_k(X, Y)$ = functions of global features

example: Feature Functions in CRF POS Tagger

Feature function f_k ที่ตำแหน่ง i สามารถขึ้นกับ (y_{i-1}, y_i, X, i) ได้อย่างอิสระ
 “Janet/NNP will/MD **back**/VB the/DT bill/NN” เมื่อ $x_i = \text{“back”}$:

```
f3743:  yi = VB    and  xi = "back"                → active (=1)
f156 :  yi = VB    and  yi-1 = MD                → active (=1)
f99732: yi = VB    and  xi-1 = "will"    and  xi+2 = "bill"    → active (=1)
```

Feature ที่ช่วยกับคำที่ไม่เคยเห็น (Unknown Words)

- Word shape: ตัวพิมพ์เล็ก→x, ตัวพิมพ์ใหญ่→X, ตัวเลข→d เช่น "I.M.F." → X.X.X., "DC10-30" → XXdd-dd
- Prefix / Suffix features: ความยาว ≤ 2-3 ตัวอักษร (เช่น ลงท้าย "-ing", "-ly", ขึ้นต้น "un-")
- ข้อดี: CRF ให้ความแม่นยำสูงกว่า HMM มากในทางปฏิบัติ เพราะรวม feature เหล่านี้ได้อย่างเป็นระบบ

สรุปเปรียบเทียบ: Rule-based vs HMM vs CRF

แนวทาง	ประเภทโมเดล	จุดเด่น	ข้อจำกัด
Rule-based (+ Brill TBL)	Knowledge-based / Hybrid	อธิบายได้ชัดเจน ไม่ต้องมีข้อมูลจำนวนมาก	เขียนกฎเองใช้แรงงานมาก ปรับ scale ยาก
HMM	Generative (สร้าง $P(W,T)$)	เรียนรู้จากข้อมูล เร็ว decode ได้ด้วย Viterbi	เพิ่ม feature พิเศษยาก, สมมติฐาน independence เข้มงวด
CRF	Discriminative (สร้าง $P(T W)$ ตรง)	ใส่ feature ได้อิสระ แม่นยำกว่า HMM มาก	train ซ้ำกว่า ต้องมี labeled data มากพอ

Sequence labelling and Name-Entity Recognition

Sequence Labeling & Named Entity Recognition



Sequence Labeling: The General Task

Given an input sequence of tokens, assign one label to each token, where labels depend on neighboring tokens as well as the token itself.

POS tagging, chunking, and NER are all instances of sequence labeling — they share the same HMM / CRF / Viterbi machinery.

input: $x_1, x_2, \dots, x_n$ $\rightarrow$ **labels:** $y_1, y_2, \dots, y_n$



NER as Sequence Labeling

Named Entity Recognition labels tokens with entity types (Person, Location, Organization) using the BIO scheme: B- begins an entity, I- continues it, O marks a non-entity.

John	lives	in	New	York
B-PER	O	O	B-LOC	I-LOC

Sequence Labeling

นิยาม: งานที่กำหนด label y_i ให้กับทุก ๆ input x_i ในลำดับ $X = x_1 \dots x_n$ โดยผลลัพธ์ $Y = y_1 \dots y_n$ มีความยาวเท่ากับ input เสมอ



POS Tagging

x_i = คำ
 y_i = หมวดคำ (NNP, VB, ...)



Named Entity Recognition

x_i = คำ
 y_i = ประเภท entity (B-PER, O, ...)



Word Segmentation

x_i = ตัวอักษร
 y_i = ขอบเขตคำ (B/I)



Chunking / Shallow Parsing

x_i = คำ
 y_i = ส่วนของวลี (NP, VP, ...)

can use tools: HMM, CRF, หรือ Neural Sequence Model (RNN/Transformer)

NER คืออะไร?

นิยาม

- งานค้นหาและระบุประเภทของ "กลุ่มคำ" (span) ที่อ้างถึงชื่อเฉพาะในข้อความ
- NER มีปัญหาเรื่อง "การแบ่งขอบเขต" (segmentation) ด้วยเพราะ entity อาจยาวหลายคำ
- Application:** Question Answering, Sentiment Analysis, Information Extraction, การเชื่อมโยงกับฐานความรู้ (Wikipedia/Knowledge Graph)

ประเภท	คำอธิบาย	ตัวอย่าง
PER (Person)	ชื่อบุคคล	สมชาย, Marie Curie
ORG (Organization)	ชื่อองค์กร/บริษัท	Google, จุฬาลงกรณ์มหาวิทยาลัย
LOC (Location)	สถานที่ทางภูมิศาสตร์	ภูเขา, แม่น้ำ, Sunshine Canyon
GPE	ประเทศ/รัฐ/เมือง	กรุงเทพ, Palo Alto

Type Ambiguity: "Washington" อาจเป็น PER (George Washington), ORG (ทีมกีฬา), หรือ LOC/GPE (เมือง) — ต้องดูบริบท!

BIO / IO / BIOES Tagging Scheme

บาง entity มีหลาย token เช่น Steve Jobs

BIO tagging แปลง NER ให้เป็น sequence labeling ที่ละคำได้

B = จุดเริ่ม entity **I = อยู่ภายใน entity** **O = ไม่ใช่ entity**

E = จุดสิ้นสุด entity **S = Single entity, มีแค่ 1 token**

(จำนวน tag = 2n+1 เมื่อ n = จำนวนประเภท entity)

Word	IO	BIO	BIOES
Jane	I-PER	B-PER	B-PER
Villanueva	I-PER	I-PER	E-PER
of	O	O	O
United	I-ORG	B-ORG	B-ORG
Airlines	I-ORG	I-ORG	I-ORG
Holding	I-ORG	I-ORG	E-ORG
discussed	O	O	O
the	O	O	O
Chicago	I-LOC	B-LOC	S-LOC
route	O	O	O

BIO Tagging บนประโยคภาษาไทย

ประโยค: “นายสมชายทำงานที่บริษัทกูเกิลในกรุงเทพ” (Mr. Somchai works at Google in Bangkok)

Word Segmentation ก่อนเสมอ (เพราะภาษาไทยไม่มีช่องว่าง) แล้วจึงทำ BIO Tagging ต่อบน token ที่ตัดคำแล้ว:

นาย	สมชาย	เรียน	ที่	ซอฟต์แวร์	ปาร์ค	ใน	กรุงเทพ
O	B-PER	O	O	B-ORG	I-ORG	O	B-LOC

- สมชาย → B-PER (จุดเริ่มต้นของชื่อบุคคล) | ซอฟต์แวร์ → B-ORG (จุดเริ่มต้นชื่อองค์กร) | ปาร์ค → B-ORG (จุดสิ้นสุดชื่อองค์กร) | กรุงเทพ → B-LOC (จุดเริ่มต้นสถานที่)
- "นาย" เป็นคำนำหน้าชื่อ (title) ไม่ใช่ส่วนหนึ่งของชื่อเฉพาะ ในตัวอย่างนี้จึงกำกับเป็น O

Evaluation NER: Precision, Recall, F1

ที่ระดับ entity

- Precision = $TP / (TP + FP)$: ในบรรดา entity ที่ระบบทายทั้งหมด ถูกต้องกี่เปอร์เซ็นต์
- Recall = $TP / (TP + FN)$: ในบรรดา entity ที่ถูกต้องจริงทั้งหมด ระบบจับได้กี่เปอร์เซ็นต์
- F1 = $2PR / (P + R)$: ค่าเฉลี่ยฮาร์โมนิกของ Precision และ Recall

		<i>gold standard labels</i>		
		gold positive	gold negative	
<i>system output labels</i>	system positive	true positive	false positive	precision = $\frac{tp}{tp+fp}$
	system negative	false negative	true negative	
		recall = $\frac{tp}{tp+fn}$		accuracy = $\frac{tp+tn}{tp+fp+tn+fn}$

Figure 4.7 A confusion matrix for visualizing how well a binary classification system performs against gold standard labels.

$$F_1 = \frac{2PR}{P + R} \tag{4.42}$$